

LAPORAN TUGAS

**WEB SERVER
(TIF 330805)**

MINGGU KE 5



ACARA 9

DIBUAT OLEH:

NAMA: CINTYA AZWA SAFRINA R.H

NIM: E41250631

GOL: A

**PROGRAM STUDI DI LUAR KAMPUS UTAMA (PSDKU) POLITEKNIK NEGERI
JEMBER**

**TEKNIK INFORMATIKA KAB. NGANJUK JL. JENDRAL GATOT SUBROTO NO
106, KAUMAN, KECAMATAN NGANJUK, KABUPATEN NGANJUK
JURUSAN TEKNOLOGI INFORMASI POLITEKNIK NEGERI JEMBER TAHUN
2026**

PENDAHULUAN

Pada praktikum sebelumnya (Acara 8), operasi CRUD data mahasiswa sudah berjalan, tetapi struktur kodenya masih sederhana. MahasiswaController membuat sendiri object MahasiswaModel dengan `new MahasiswaModel()`, data mahasiswa dikirim dalam bentuk array biasa, dan validasi hanya memeriksa NIM dan nama tidak kosong. Controller menjadi bergantung langsung pada cara data disimpan, sehingga sulit dikembangkan dan diuji.

Praktikum ini memecah tanggung jawab tersebut dengan konsep pemrograman berorientasi objek. Pertama, **Entity** Mahasiswa merepresentasikan satu data mahasiswa sebagai object. Properti dibuat `private` (encapsulation) dan hanya dapat diakses melalui method getter. Method setter menyertakan validasi dan melempar `InvalidArgumentException` apabila data tidak valid, misalnya NIM yang bukan angka.

Kedua, **Repository** MahasiswaRepository menampung seluruh query SQL ke database, sehingga Controller tidak lagi menulis SQL. Method Repository menerima dan mengembalikan object Mahasiswa, bukan array mentah.

Ketiga, **Dependency Injection**: sebuah class tidak membuat sendiri object yang dibutuhkannya, melainkan menerimanya dari luar melalui constructor. MahasiswaController menerima MahasiswaRepository, dan MahasiswaRepository menerima object Database. Hubungan "memiliki" antar object ini disebut **Object Composition**. Seluruh object dirangkai di satu tempat yang disebut **Composition Root**, yaitu file `routes/web.php`. Agar Database dapat di-inject, class tersebut tetap berpola **Singleton** tetapi kini mengembalikan object Database, bukan langsung object PDO.

IMPLEMENTASI

1. Menyiapkan folder Acara_9

Project Sistem Informasi Akademik dari Acara 8 digunakan sebagai dasar pengerjaan pada folder Acara_9. Perubahan utama: MahasiswaModel.php dihapus dan diganti oleh folder Entities dan Repositories. ProdiModel dan MatakuliahModel tidak diubah.

```

app
├── Controllers  MahasiswaController.php (diubah)
├── Core         Database.php (diubah)
├── Entities     Mahasiswa.php (BARU)
├── Middleware
├── Models       ProdiModel.php, MatakuliahModel.php
├── Repositories MahasiswaRepository.php (BARU)
└── Views

```

2. Mengubah class Database agar dapat di-inject

Database::getInstance() sekarang mengembalikan object Database (bukan PDO). Class tetap Singleton: constructor private sehingga hanya ada satu instance koneksi. Koneksi PDO diambil melalui method getConnection().

```

<?php
class Database
{
    private static ?Database $instance = null;
    private PDO $pdo;

    // Constructor private -> object Database hanya bisa dibuat lewat getInstance()
    private function __construct()
    {
        $config = require __DIR__ . '/../../config/database.php';
        $dsn = "mysql:host={$config['host']};dbname={$config['dbname']};charset={$config['charset']}";

        $this->pdo = new PDO($dsn, $config['username'], $config['password'], [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_EMULATE_PREPARES => false,
        ]);
    }

    // Tetap satu-satunya instance (Singleton), tapi sekarang mengembalikan
    // OBJEK Database, bukan langsung PDO -- supaya bisa di-inject via constructor
    public static function getInstance(): self
    {
        if (self::$instance === null) {
            self::$instance = new self();
        }
        return self::$instance;
    }

    public function getConnection(): PDO
    {
        return $this->pdo;
    }
}

```

3. Membuat Entity Mahasiswa

File app/Entities/Mahasiswa.php dibuat. Properti bersifat private, nilai diberikan melalui constructor yang memanggil setter, dan setter melakukan validasi: NIM harus angka, nama tidak boleh kosong, prodi harus dipilih, angkatan antara 2000 sampai tahun berjalan, dan status harus aktif, cuti, atau lulus. Method statis fromArray() mengubah satu baris hasil query menjadi object Mahasiswa.

```
<?php
class Mahasiswa
{
    private ?int $id;
    private string $nim;
    private string $nama;
    private string $email;
    private int $prodiId;
    private int $angkatan;
    private string $status;

    public function __construct(
        string $nim,
        string $nama,
        string $email,
        int $prodiId,
        int $angkatan,
        string $status = 'aktif',
        ?int $id = null
    ) {
        $this->setNim($nim);
        $this->setNama($nama);
        $this->setEmail($email);
        $this->setProdiId($prodiId);
        $this->setAngkatan($angkatan);
        $this->setStatus($status);
        $this->id = $id;
    }

    // ===== GETTER =====

    public function getId(): ?int
    {
        return $this->id;
    }

    public function getNim(): string
    {
        return $this->nim;
    }

    public function getNama(): string
    {
        return $this->nama;
    }

    public function getEmail(): string
    {
        return $this->email;
    }

    public function getProdiId(): int
    {
        return $this->prodiId;
    }
}
```

```

public function getAngkatan(): int
{
    return $this->angkatan;
}

public function getStatus(): string
{
    return $this->status;
}

// ===== SETTER (dengan validasi) =====

public function setNim(string $nim): void
{
    // Validasi: NIM harus berupa angka
    if (!ctype_digit($nim)) {
        throw new InvalidArgumentException("NIM harus berupa angka, diberikan: '{$nim}'");
    }
    $this->nim = $nim;
}

public function setName(string $nama): void
{
    // Validasi: nama tidak boleh kosong
    if (trim($nama) === '') {
        throw new InvalidArgumentException("Nama mahasiswa tidak boleh kosong");
    }
    $this->nama = trim($nama);
}

public function setEmail(string $email): void
{
    $this->email = trim($email);
}

public function setProdiId(int $prodiId): void
{
    if ($prodiId <= 0) {
        throw new InvalidArgumentException("Prodi harus dipilih");
    }
    $this->prodiId = $prodiId;
}

public function setAngkatan(int $angkatan): void
{
    $tahunSekarang = (int) date('Y');
    if ($angkatan < 2000 || $angkatan > $tahunSekarang) {
        throw new InvalidArgumentException("Angkatan tidak valid: {$angkatan}");
    }
    $this->angkatan = $angkatan;
}

public function setStatus(string $status): void
{
    $validStatus = ['aktif', 'cuti', 'lulus'];
    if (!in_array($status, $validStatus, true)) {
        throw new InvalidArgumentException("Status harus salah satu dari: " . implode(', ', $validStatus));
    }
    $this->status = $status;
}

// ===== HELPER =====

```

```
// Membuat object Mahasiswa dari baris hasil query database
public static function fromArray(array $data): self
{
    return new self(
        $data['nim'],
        $data['nama'],
        $data['email'],
        (int) $data['prodi_id'],
        (int) $data['angkatan'],
        $data['status'],
        isset($data['id']) ? (int) $data['id'] : null
    );
}
}
```

4. Membuat MahasiswaRepository

File `app/Repositories/MahasiswaRepository.php` dibuat. Class ini menerima object Database melalui constructor, lalu menyediakan method `all()`, `find()`, `create()`, `update()`, dan `delete()`. Semua query memakai prepared statement.

```
<?php
require_once __DIR__ . '/../Core/Database.php';
require_once __DIR__ . '/../Entities/Mahasiswa.php';

class MahasiswaRepository
{
    private PDO $pdo;

    // Constructor Dependency Injection: menerima objek Database,
    // bukan membuat koneksi sendiri di dalam class ini.
    public function __construct(Database $database)
    {
        $this->pdo = $database->getConnection();
    }

    // Mengembalikan array of Mahasiswa (Object Composition: Repository "memiliki" PDO)
    public function all(?string $keyword = null): array
    {
        $sql = "SELECT mahasiswa.*, prodi.nama AS prodi_nama
                FROM mahasiswa
                JOIN prodi ON mahasiswa.prodi_id = prodi.id";

        if ($keyword !== null && $keyword !== '') {
            $sql .= " WHERE mahasiswa.nama LIKE :kw1 OR mahasiswa.nim LIKE :kw2";
        }
        $sql .= " ORDER BY mahasiswa.id ASC";

        $stmt = $this->pdo->prepare($sql);

        if ($keyword !== null && $keyword !== '') {
            $stmt->execute(['kw1' => '%' . $keyword . '%', 'kw2' => '%' . $keyword . '%']);
        } else {
            $stmt->execute();
        }

        $rows = $stmt->fetchAll();

        // Konversi tiap baris menjadi object Mahasiswa + simpan nama prodi terpisah
        $result = [];
        foreach ($rows as $row) {
            $result[] = [
                'mahasiswa' => Mahasiswa::fromArray($row),
                'prodi_nama' => $row['prodi_nama'],
            ];
        }
    }
}
```

```

    ];
    }
    return $result;
}

public function find(int $id): ?Mahasiswa
{
    $stmt = $this->pdo->prepare("SELECT * FROM mahasiswa WHERE id = :id");
    $stmt->execute(['id' => $id]);
    $row = $stmt->fetch();

    return $row ? Mahasiswa::fromArray($row) : null;
}

public function create(Mahasiswa $mahasiswa): void
{
    $stmt = $this->pdo->prepare(
        "INSERT INTO mahasiswa (nim, nama, email, prodi_id, angkatan, status)
        VALUES (:nim, :nama, :email, :prodi_id, :angkatan, :status)"
    );
    $stmt->execute([
        'nim' => $mahasiswa->getNim(),
        'nama' => $mahasiswa->getNama(),
        'email' => $mahasiswa->getEmail(),
        'prodi_id' => $mahasiswa->getProdiId(),
        'angkatan' => $mahasiswa->getAngkatan(),
        'status' => $mahasiswa->getStatus(),
    ]);
}

public function update(int $id, Mahasiswa $mahasiswa): void
{
    $stmt = $this->pdo->prepare(
        "UPDATE mahasiswa
        SET nim = :nim, nama = :nama, email = :email,
            prodi_id = :prodi_id, angkatan = :angkatan, status = :status
        WHERE id = :id"
    );
    $stmt->execute([
        'nim' => $mahasiswa->getNim(),
        'nama' => $mahasiswa->getNama(),
        'email' => $mahasiswa->getEmail(),
        'prodi_id' => $mahasiswa->getProdiId(),
        'angkatan' => $mahasiswa->getAngkatan(),
        'status' => $mahasiswa->getStatus(),
        'id' => $id,
    ]);
}

public function delete(int $id): void
{
    $stmt = $this->pdo->prepare("DELETE FROM mahasiswa WHERE id = :id");
    $stmt->execute(['id' => $id]);
}
}

```

5. Mengubah MahasiswaController

Controller kini menerima MahasiswaRepository melalui constructor dan tidak lagi memakai `new MahasiswaModel()`. Pada `store()` dan `update()`, input dibungkus menjadi object Mahasiswa. Jika ada data tidak valid, setter melempar `InvalidArgumentException` yang ditangkap dengan `try/catch`, lalu pesan error dikirim ke form.

```

<?php
require_once __DIR__ . '/../Repositories/MahasiswaRepository.php';

```

```

require_once __DIR__ . '/../Entities/Mahasiswa.php';
require_once __DIR__ . '/../Models/ProdiModel.php';

class MahasiswaController
{
    private MahasiswaRepository $repo;

    // Constructor Dependency Injection: Controller menerima Repository dari luar,
    // bukan membuat sendiri (dulu di Acara 8 Controller yang bikin "new MahasiswaModel()").
    public function __construct(MahasiswaRepository $repo)
    {
        $this->repo = $repo;
    }

    private function base(): string
    {
        return str_replace('\\', '/', dirname($_SERVER['SCRIPT_NAME']));
    }

    public function index(): void
    {
        $keyword = trim($_GET['q'] ?? '');
        $daftarMahasiswa = $this->repo->all($keyword !== '' ? $keyword : null);

        require_once __DIR__ . '/../Views/mahasiswa/index.php';
    }

    public function create(): void
    {
        $prodiModel = new ProdiModel();
        $daftarProdi = $prodiModel->all();

        require_once __DIR__ . '/../Views/mahasiswa/create.php';
    }

    public function store(): void
    {
        try {
            // Getter/setter pada class Mahasiswa otomatis memvalidasi input di sini
            $mahasiswa = new Mahasiswa(
                trim($_POST['nim'] ?? ''),
                trim($_POST['nama'] ?? ''),
                trim($_POST['email'] ?? ''),
                (int) ($_POST['prodi_id'] ?? 0),
                (int) ($_POST['angkatan'] ?? date('Y')),
                $_POST['status'] ?? 'aktif'
            );

            $this->repo->create($mahasiswa);
            header("Location: {$this->base()}/mahasiswa");
            exit;
        } catch (InvalidArgumentException $e) {
            $pesan = urlencode($e->getMessage());
            header("Location: {$this->base()}/mahasiswa/create?error={$pesan}");
            exit;
        }
    }

    public function edit(int $id): void
    {
        $mahasiswa = $this->repo->find($id);

        if ($mahasiswa === null) {
            http_response_code(404);
            echo "404 - Mahasiswa tidak ditemukan";
        }
    }
}

```



```

        return;
    }

    $prodiModel = new ProdiModel();
    $daftarProdi = $prodiModel->all();

    require_once __DIR__ . '/../Views/mahasiswa/edit.php';
}

public function update(int $id): void
{
    try {
        $mahasiswa = new Mahasiswa(
            trim($_POST['nim'] ?? ''),
            trim($_POST['nama'] ?? ''),
            trim($_POST['email'] ?? ''),
            (int) ($_POST['prodi_id'] ?? 0),
            (int) ($_POST['angkatan'] ?? date('Y')),
            $_POST['status'] ?? 'aktif',
            $id
        );

        $this->repo->update($id, $mahasiswa);
        header("Location: {$this->base()}/mahasiswa");
        exit;
    } catch (InvalidArgumentException $e) {
        $pesan = urlencode($e->getMessage());
        header("Location: {$this->base()}/mahasiswa/{$id}/edit?error={$pesan}");
        exit;
    }
}

public function destroy(int $id): void
{
    $this->repo->delete($id);
    header("Location: {$this->base()}/mahasiswa");
    exit;
}
}

```

6. Merangkai dependency pada routes/web.php (Composition Root)

Karena MahasiswaController membutuhkan Repository, dan Repository membutuhkan Database, objek dirangkai berurutan pada routes/web.php. Controller lain masih dibuat langsung karena belum termasuk cakupan refactor Acara 9.

```

// =====
// Composition Root: di sinilah semua dependency dirangkai.
// MahasiswaController butuh MahasiswaRepository,
// MahasiswaRepository butuh Database.
// Controller lain masih dibuat langsung (di luar cakupan refactor Acara 9).
// =====
if ($controllerName === 'MahasiswaController') {
    $database = Database::getInstance(); // 1. Database (Singleton)
    $repository = new MahasiswaRepository($database); // 2. Repository menerima
Database
    $controller = new MahasiswaController($repository); // 3. Controller menerima
Repository
} else {
    $controller = new $controllerName();
}

```

HASIL PENGUJIAN

Seluruh fitur CRUD mahasiswa diuji melalui browser untuk memastikan hasil refactoring tidak mengubah perilaku aplikasi.

No	Skenario	Hasil yang Diharapkan	Hasil
1	Membuka halaman daftar mahasiswa	Data tampil dari database melalui Repository	Sesuai
2	Menambah mahasiswa dengan data valid	Data tersimpan dan muncul di daftar	Sesuai
3	Menambah mahasiswa dengan NIM berisi huruf	Muncul pesan error dari setter Entity	Sesuai
4	Mengubah data mahasiswa	Data berubah pada daftar	Sesuai
5	Menghapus data mahasiswa	Data hilang dari daftar	Sesuai

a. Halaman daftar mahasiswa

**Sistem Informasi Akademik**
Daftar Mahasiswa

NIM	Nama	Email	Prodi	Angkatan	Status	Aksi
2401001	Budi Santoso	budi@email.com	Teknik Informatika	2024	cuti	Edit Hapus
2401002	Ani Wijaya	ani@email.com	Teknik Informatika	2024	aktif	Edit Hapus
2402001	Citra Lestari	citra@email.com	Sistem Informasi	2024	aktif	Edit Hapus
23004	Dewi	dewi@gmail.com	Teknik Informatika	2026	aktif	Edit Hapus

b. Tambah data valid

**Sistem Informasi Akademik**
Tambah Mahasiswa

NIM

09086543

Nama

admin

Email

cintyaazw40@gmail.com

Program Studi

Teknik Informatika

Angkatan

2026

Status

Aktif

Daftar Mahasiswa

Cari nama atau NIM... Cari + Tambah Mahasiswa

NIM	Nama	Email	Prodi	Angkatan	Status	Aksi
2401001	Budi Santoso	budi@email.com	Teknik Informatika	2024	cuti	Edit Hapus
2401002	Ani Wijaya	ani@email.com	Teknik Informatika	2024	aktif	Edit Hapus
2402001	Citra Lestari	citra@email.com	Sistem Informasi	2024	aktif	Edit Hapus
23004	Dewi	dewi@gmail.com	Teknik Informatika	2026	aktif	Edit Hapus
09086543	admin	cintyaazw40@gmail.com	Teknik Informatika	2026	aktif	Edit Hapus

Kembali ke Dashboard

c. Validasi dari Entity

Ketika NIM diisi huruf, setter `setNim()` melempar `InvalidArgumentException`. Pesan error ditangkap Controller dan ditampilkan pada form.



Sistem Informasi Akademik

Tambah Mahasiswa

NIM harus berupa angka, diberikan: '0908654333afddgh'

d. Ubah dan hapus data

Teknik Informatika

Angkatan

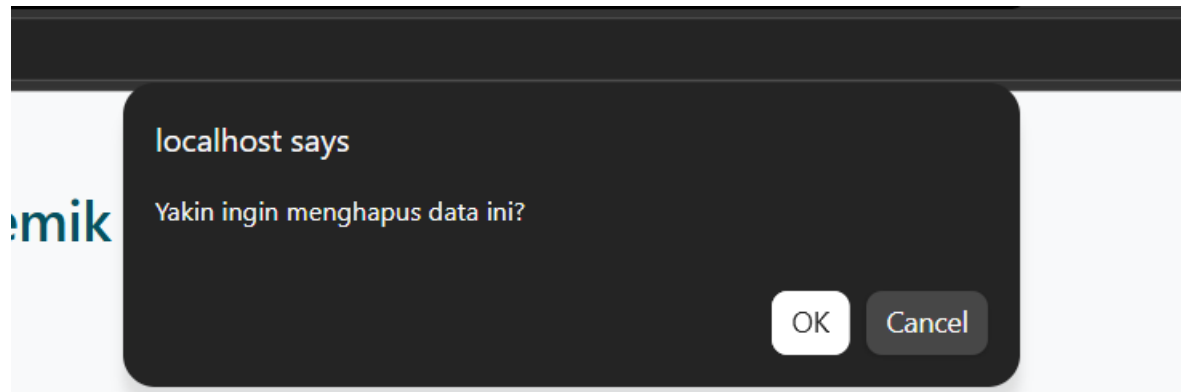
Aktif
Cuti
Lulus

Lulus

Update

Batal

NIM	Nama	Email	Prodi	Angkatan	Status	Aksi
2401001	Budi Santoso	budi@email.com	Teknik Informatika	2024	lulus	Edit Hapus
2401002	Ani Wiiava	ani@email.com	Teknik Informatika	2024	aktif	Edit Hapus



ANALISIS

Dibandingkan Acara 8, `MahasiswaController` tidak lagi membuat `new MahasiswaModel()` dan tidak lagi memuat query SQL. Seluruh SQL berada di `MahasiswaRepository`, sehingga perubahan cara penyimpanan data cukup dilakukan di `Repository` tanpa menyentuh `Controller`.

Class `Mahasiswa` menerapkan encapsulation: properti `private` hanya dapat diubah lewat setter yang memvalidasi nilainya. Validasi pada Acara 8 yang hanya memeriksa NIM dan nama tidak kosong kini diperluas, yaitu NIM harus angka, prodi harus dipilih, angkatan harus valid, dan status harus sesuai daftar. Data yang tidak valid tidak pernah menjadi object, sehingga tidak sampai ke database.

Dependency Injection membuat ketergantungan antar class terlihat jelas pada constructor: `Controller` membutuhkan `Repository`, dan `Repository` membutuhkan `Database`. Perangkaian object dipusatkan di `Composition Root` (`routes/web.php`), sehingga class lain tidak perlu tahu cara membuat dependency-nya. Dengan `Singleton` yang mengembalikan object `Database`, koneksi tetap hanya satu tetapi dapat di-inject.

Secara keseluruhan, refactoring ini membuat kode lebih terstruktur, mudah dipelihara, dan siap dikembangkan, misalnya dengan menambahkan lapisan `Service` pada praktikum berikutnya.

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. Entity merepresentasikan satu data (mahasiswa) sebagai object dengan properti `private` dan `getter/setter`, sehingga data terlindungi dan tervalidasi.
2. Setter pada Entity melempar `InvalidArgumentException` untuk data tidak valid sehingga data salah tidak sampai tersimpan.
3. Repository memisahkan seluruh query SQL dari Controller dan bekerja dengan object Mahasiswa.
4. Dependency Injection melalui constructor membuat class tidak membuat dependency-nya sendiri sehingga ketergantungan lebih jelas dan mudah diganti.
5. Composition Root pada `routes/web.php` menjadi satu-satunya tempat perangkaian object Database, Repository, dan Controller.
6. Singleton yang mengembalikan object Database menjaga koneksi tetap satu sekaligus memungkinkan injeksi dependency.

DAFTAR PUSTAKA

1. Politeknik Negeri Jember. 2026. Modul Praktikum Workshop SI Web Server (TIF330805), Acara 9.
2. The PHP Group. PHP Manual: Classes and Objects.
<https://www.php.net/manual/en/language.oop5.php>
3. The PHP Group. PHP Manual: PHP Data Objects (PDO).
<https://www.php.net/manual/en/book.pdo.php>